

S40 · Actividad 3 — Evidencia de Aprendizaje 3

Ingeniería Web

CreditSmart — Integración con Backend Firebase (Firestore)

Documento técnico: persistencia de datos en la nube con Cloud Firestore, operaciones CRUD (addDoc, getDocs), consultas con where y orderBy, comunicación asíncrona, manejo de errores y variables de entorno, sobre la arquitectura hexagonal construida en las Actividades 1 y 2.

Integrantes

Jeremy Ivan Pedraza Hernández
Jossy Esteban Paternina Julio

Docente

Jorge Armando Julio

Curso y NRC

Ingeniería Web — PREICA2602B010133

Ingeniería de Software y Datos

Facultad de Ingeniería y Ciencias Agropecuarias
Institución Universitaria Digital de Antioquia
2026-S2

29 de Septiembre 2026

Tabla de contenido

Contenido

S40 · Actividad 3 — Evidencia de Aprendizaje 3.....	1
Ingeniería Web.....	1
CreditSmart — Integración con Backend Firebase (Firestore)	1
Tabla de contenido	2
1. Introducción	4
2. Objetivo.....	4
2.1 Objetivo general	4
2.2 Objetivos específicos.....	4
3. Alcance y requerimientos	5
3.1 Requerimientos funcionales.....	5
3.2 Requerimientos no funcionales.....	5
4. Configuración de Firebase	6
4.1 Proyecto y app web	6
4.2 Habilitar Cloud Firestore	6
4.3 Variables de entorno	6
5. Firebase como un adaptador más.....	7
5.1 La regla de dependencia no cambió	7
5.2 El único archivo que cambió para ir a la nube	7
5.3 Contratos verificados al arrancar	8
6. Modelo de datos en Firestore.....	8
7. Operación READ — listar créditos	8
8. Operación CREATE — guardar solicitudes	9
9. Consultas y filtros.....	11
10. Manejo de errores	12
11. Seguridad — variables de entorno	13
12. Verificación	13
12.1 Comprobaciones automáticas	13
12.2 Comprobación de la conexión con Firestore	13

12.3 Comprobación en el navegador.....	13
13. Cumplimiento de la rúbrica.....	16
13.1 Observaciones del estudiante.....	¡Error! Marcador no definido.
14. Entregables.....	17
14.1 Repositorio	17
14.2 Historial de commits	17
15. Conclusiones	18
16. Referencias.....	18

1. Introducción

CreditSmart es una plataforma web para consultar, simular y solicitar productos de crédito. En la Actividad 1 se construyó con JavaScript estándar sobre una arquitectura hexagonal; en la Actividad 2 su interfaz se reescribió en React sin tocar el núcleo. En ambas, los datos vivían solo en memoria y en el navegador: al cerrar la pestaña, las solicitudes se perdían.

Esta Actividad 3 conecta CreditSmart con Firebase, la plataforma de Google que ofrece backend como servicio (BaaS), y en particular con Cloud Firestore, su base de datos NoSQL. A partir de ahora el catálogo de productos y las solicitudes de crédito se almacenan en la nube, persisten entre sesiones y pueden consultarse, crearse y filtrarse desde cualquier dispositivo.

La tesis del documento es la misma que sostuvo la migración a React: cambiar de fuente de datos no fue una reescritura, sino la sustitución de un adaptador. La arquitectura hexagonal define puertos (interfaces) que el dominio conoce y que la infraestructura implementa. Pasar de datos estáticos y localStorage a Firestore significó escribir dos adaptadores nuevos y cambiar unas líneas del contenedor de dependencias. El dominio, los casos de uso y las vistas no se enteraron.

2. Objetivo

2.1 Objetivo general

Aplicar los conceptos de persistencia de datos, base de datos NoSQL (Firestore), operaciones CRUD y comunicación asíncrona para integrar CreditSmart con Firebase, permitiendo que los datos se almacenen en la nube, se consulten y persistan entre sesiones, sin degradar la separación entre la interfaz, las reglas del negocio y la tecnología de persistencia.

2.2 Objetivos específicos

- Configurar un proyecto de Firebase, registrar una app web y habilitar Cloud Firestore como base de datos NoSQL.
- Instalar y configurar el SDK modular de Firebase en el proyecto React, leyendo las credenciales desde variables de entorno de Vite.
- Implementar la operación READ del catálogo con getDocs, con estado de carga y manejo de errores, mapeando cada documento a la entidad del dominio.
- Implementar la operación CREATE de solicitudes con addDoc, validando antes de guardar, limpiando el formulario y redirigiendo tras el éxito.
- Implementar consultas con where y orderBy en una página «Mis solicitudes» que filtra por el correo del solicitante.

- Manejar errores con try-catch en todas las operaciones y mostrar mensajes claros al usuario, incluida la prueba de desconexión de red.
- Proteger las credenciales con variables de entorno, dejando .env fuera del control de versiones y documentando las claves en .env.example.
- Demostrar, con evidencia medible, que la persistencia en la nube fue un cambio de adaptador y no una reescritura del sistema.

3. Alcance y requerimientos

3.1 Requerimientos funcionales

ID	Requerimiento	Dónde se implementa
RF-1	El catálogo de productos se lee desde Firestore (getDocs).	FirestoreCreditProductRepository
RF-2	Las solicitudes se guardan permanentemente en Firestore (addDoc).	FirestoreCreditApplicationRepository.save
RF-3	Se pueden consultar las solicitudes de un correo (where + orderBy).	findByApplicantEmail · ListMyApplicationsUseCase
RF-4	La interfaz muestra un estado de carga mientras lee de la nube.	useCreditProducts · useMyApplications
RF-5	Los errores de red se muestran al usuario con un mensaje claro.	try-catch + Result + avisos (toasts)
RF-6	Los datos persisten aunque se cierre el navegador.	Cloud Firestore
RF-7	Tras registrar una solicitud, se redirige a «Mis solicitudes».	ApplicationPage → MyApplicationsPage

Tabla 1. Requerimientos funcionales de la Actividad 3

3.2 Requerimientos no funcionales

- Seguridad: las credenciales de Firebase no se versionan; entran por variables de entorno (import.meta.env).
- Resiliencia: si Firebase no está configurado o la red falla, la aplicación degrada a datos locales en lugar de romperse.
- Asincronía: toda operación con Firestore es asíncrona (async/await) y no bloquea la interfaz; un tope de tiempo evita cuelgues.
- Compatibilidad con la arquitectura: la persistencia en la nube no viola la regla de dependencia ni obliga a tocar el dominio.
- Consumo dentro del plan gratuito de Firestore (50K lecturas y 20K escrituras al día).

4. Configuración de Firebase

4.1 Proyecto y app web

Se creó un proyecto en la consola de Firebase y, dentro de él, una aplicación web. La consola entrega un objeto de configuración (firebaseConfig) con las claves del proyecto: apiKey, authDomain, projectId, storageBucket, messagingSenderId y appId.

4.2 Habilitar Cloud Firestore

En Compilación → Firestore Database se creó la base de datos en modo de prueba, que abre las reglas de lectura y escritura durante 30 días —suficiente para la entrega—. Al ejecutar la aplicación por primera vez, la colección de productos se siembra automáticamente con el catálogo base.

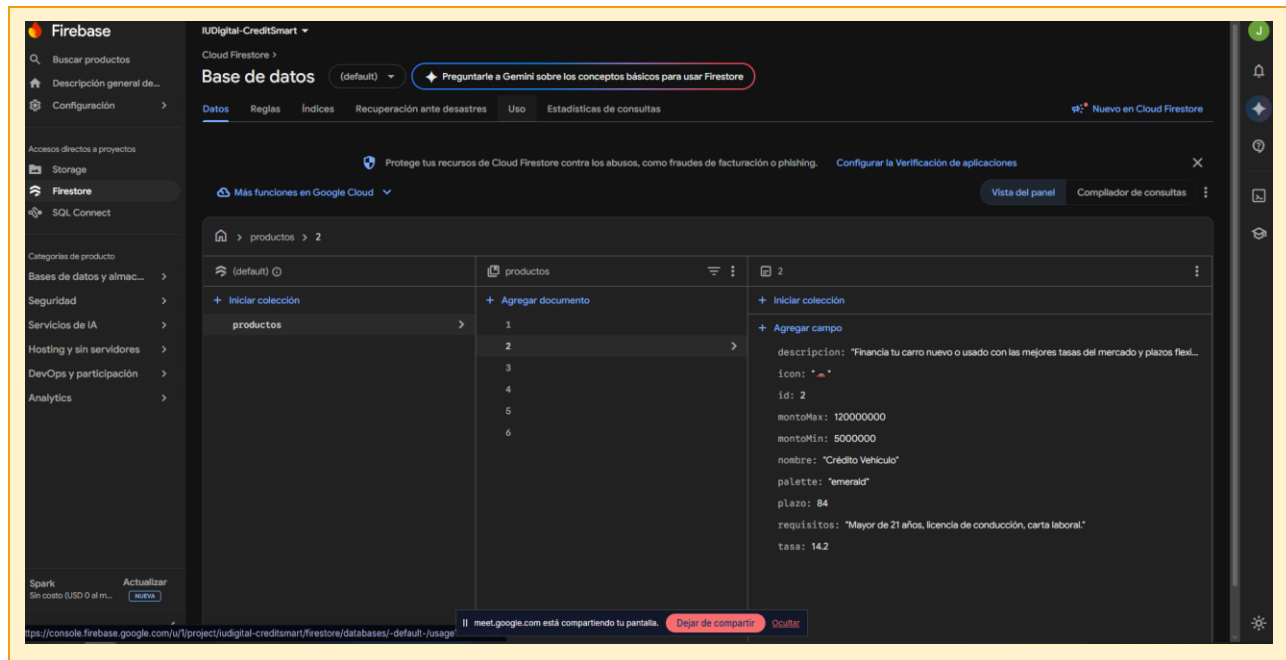


Figura 1. Consola de Firebase con las colecciones de Firestore

4.3 Variables de entorno

La configuración no se escribe en el código: se lee de variables de entorno de Vite, que solo expone al navegador las que llevan el prefijo VITE_. El composition root (config/dependencies.js) las lee y se las inyecta por constructor a FirebaseClient; ninguna otra clase lee configuración global.

```
# .env (este archivo está en .gitignore: nunca se sube al repositorio)
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=iudigital-creditsmart.firebaseio.com
VITE_FIREBASE_PROJECT_ID=iudigital-creditsmart
VITE_FIREBASE_STORAGE_BUCKET=iudigital-creditsmart.firebaseio.com
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
```

Figura 2. Archivo .env con las credenciales (valores recortados)

5. Firebase como un adaptador más

5.1 La regla de dependencia no cambió

El dominio define puertos (interfaces) y la infraestructura los implementa. Las flechas apuntan siempre hacia el núcleo: la presentación depende de la aplicación, la aplicación del dominio, y la infraestructura también del dominio. Firestore es un detalle de infraestructura que el dominio nunca conoce.



Figura 3. La infraestructura implementa los puertos del dominio; la dependencia apunta al núcleo

5.2 El único archivo que cambió para ir a la nube

Pasar de datos estáticos y localStorage a Firestore fue cambiar el cableado en config/dependencies.js: registrar el cliente de Firebase y apuntar los dos repositorios a sus adaptadores de Firestore. Ni el dominio, ni los casos de uso, ni un solo componente se modificaron.

```
container.register('firebaseClient', (c) =>
  new FirebaseClient({ config: readFirebaseConfig(), logger: c.resolve('logger') }));

container.register('productRepository', (c) =>
  assertImplements(new FirestoreCreditProductRepository({
    firebaseClient: c.resolve('firebaseClient'), logger: c.resolve('logger'),
  }), ICreditProductRepository));

container.register('applicationRepository', (c) =>
  assertImplements(new FirestoreCreditApplicationRepository({
    firebaseClient: c.resolve('firebaseClient'),
    idGenerator: c.resolve('idGenerator'), logger: c.resolve('logger'),
  }), ICreditApplicationRepository));
```

Figura 4. config/dependencies.js: el cambio de adaptador vive en un solo lugar

5.3 Contratos verificados al arrancar

El puerto de solicitudes ganó un método, `findByApplicantEmail`, para la consulta por correo. Se añadió al contrato del dominio y a las dos implementaciones (Firestore y la de `localStorage`, que queda como respaldo). `assertImplements` comprueba en el arranque que cada adaptador cumple su contrato: un método faltante rompe la carga de la página, no una navegación posterior.

6. Modelo de datos en Firestore

Firestore es una base de datos NoSQL orientada a documentos: los datos se organizan en colecciones de documentos, cada uno un conjunto de campos. CreditSmart usa dos colecciones.

Colección	Documento	Campos principales	Operaciones
productos	un producto de crédito	nombre, descripcion, montoMin, montoMax, tasa, plazo, requisitos, icon, palette	getDocs, setDoc (siembra)
solicitudes	una solicitud radicada	id, status, reference, createdAt, applicantEmail, applicant{}, requestedCredit{}, employmentInfo{}	addDoc, getDocs, where, orderBy

Tabla 2. Las dos colecciones de Firestore

El documento de solicitud guarda la forma serializada de la entidad más dos campos de nivel superior: `applicantEmail` (en minúsculas, para filtrar) y `createdAt` como fecha (para ordenar). Una `factory` reconstruye la entidad del dominio y sus `value objects` al leer de Firestore, validando en la frontera.

7. Operación READ — listar créditos

El catálogo se lee de Firestore con `getDocs`. Si la colección está vacía, se siembra en segundo plano con el catálogo base y se devuelven los datos locales de inmediato, para que la pantalla nunca quede en espera. Cada documento se mapea a la entidad del dominio conservando su identificador.

```
const snapshot = await getDocs(collection(this.#db, 'productos'));
return snapshot.docs.map((d) => CreditProductFactory.fromRaw(d.data()));
```

Figura 5. Lectura del catálogo con `getDocs` y mapeo a entidades

El hook `useCreditProducts` invoca el caso de uso, desenvuelve el `Result` y expone a la página un estado plano con `isLoading` y `error`. La página no sabe que existe un `Result`, un repositorio ni Firestore.



Figura 6. Catálogo de CreditSmart cargando los productos desde Firestore.

8. Operación CREATE — guardar solicitudes

El formulario de solicitud, controlado y validado en tiempo real con las reglas del dominio, envía la solicitud al caso de uso, que construye la entidad y la persiste con `addDoc`. Tras el éxito, el formulario se limpia y la confirmación muestra el número de radicado, con un enlace directo a «Mis solicitudes».

```
const payload = {
  ...application.toJSON(),
  applicantEmail: application.applicant.email,
  createdAt: application.createdAt,
};
await addDoc(collection(this.#db, 'solicitudes'), payload);
```

Figura 7. Guardado de la solicitud con `addDoc`

CreditSmart Inicio Productos Simulador **Solicitar** Mis solicitudes Ayuda Iniciar solicitud

Inicio / Solicitar

Solicitud de crédito

Completa tus datos para registrar la solicitud.

1 Tus datos 2 Crédito y finanzas 3 Datos laborales 4 Confirmación

Tus datos personales
Comencemos con tu información básica.

Nombre completo * Ej: Juan Carlos Pérez

Cédula * Ej: 1234567890

Correo electrónico * Ej: juan@correo.com

Teléfono * Ej: 3001234567

[Limpiar formulario](#) [Continuar →](#)

Tu información está protegida

Usamos prácticas de seguridad para proteger la información que ingresas en este formulario.

- Los datos se usan solo para evaluar esta solicitud
- No se comparten con terceros
- Puedes limpiar el formulario en cualquier momento

Los campos marcados con * son obligatorios. La validación se ejecuta mientras escribes, con las mismas reglas que se aplican al enviar.

Figura 8. Formulario de solicitud por pasos, con validación en vivo.

CreditSmart Inicio Productos Simulador **Solicitar** Mis solicitudes Ayuda Iniciar solicitud

Inicio / Solicitar

Tus datos Crédito y finanzas Datos laborales **Confirmación**

Solicitud registrada

Tu solicitud fue registrada correctamente. Guarda este número para hacer seguimiento.

Número de radicado
CS-DF8B5131

[Ver mis solicitudes](#) [Volver al inicio](#) [Registrar otra solicitud](#)

CreditSmart Más que crédito, progreso para ti. «Hoy es un buen momento para construir el mañana.»

¡Listo!
Solicitud registrada correctamente para Jeremy.

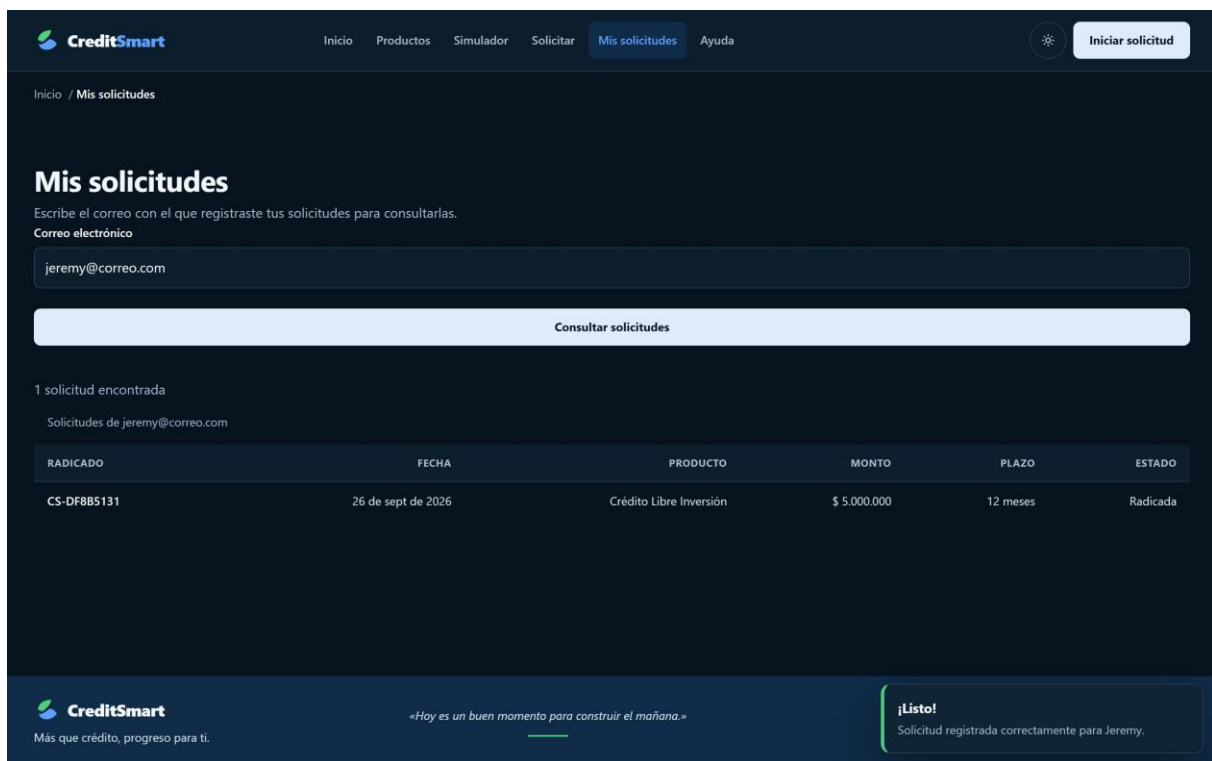
Figura 9. Confirmación con el número de radicado y el enlace a «Mis solicitudes».

9. Consultas y filtros

La página «Mis solicitudes» consulta las solicitudes de un correo combinando `where` y `orderBy`: filtra por `applicantEmail` y ordena por fecha descendente. Esa combinación requiere un índice compuesto que Firestore ofrece crear con un clic la primera vez; mientras no exista, la aplicación degrada a filtrar por `where` y ordenar en el cliente, así que sigue funcionando.

```
const q = query(
  collection(this.#db, 'solicitudes'),
  where('applicantEmail', '=', email),
  orderBy('createdAt', 'desc'),
);
const snapshot = await getDocs(q);
```

Figura 10. Consulta con `where` + `orderBy`



CreditSmart Inicio Productos Simulador Solicitar **Mis solicitudes** Ayuda ⚙️ Iniciar solicitud

Inicio / Mis solicitudes

Mis solicitudes

Escribe el correo con el que registraste tus solicitudes para consultarlas.

Correo electrónico

jeremy@correo.com

Consultar solicitudes

1 solicitud encontrada

Solicitudes de jeremy@correo.com

RADICADO	FECHA	PRODUCTO	MONTO	PLAZO	ESTADO
CS-DF885131	26 de sept de 2026	Crédito Libre Inversión	\$ 5.000.000	12 meses	Radicada

CreditSmart Más que crédito, progreso para ti. «Hoy es un buen momento para construir el mañana.» **¡Listo!**
Solicitud registrada correctamente para Jeremy.

Figura 11. «Mis solicitudes»: solicitudes de un correo, consultadas con `where` y `orderBy`.

Antes de buscar, la página muestra solo el campo de correo; durante la lectura, un estado de carga; y si no hay resultados, un aviso claro.

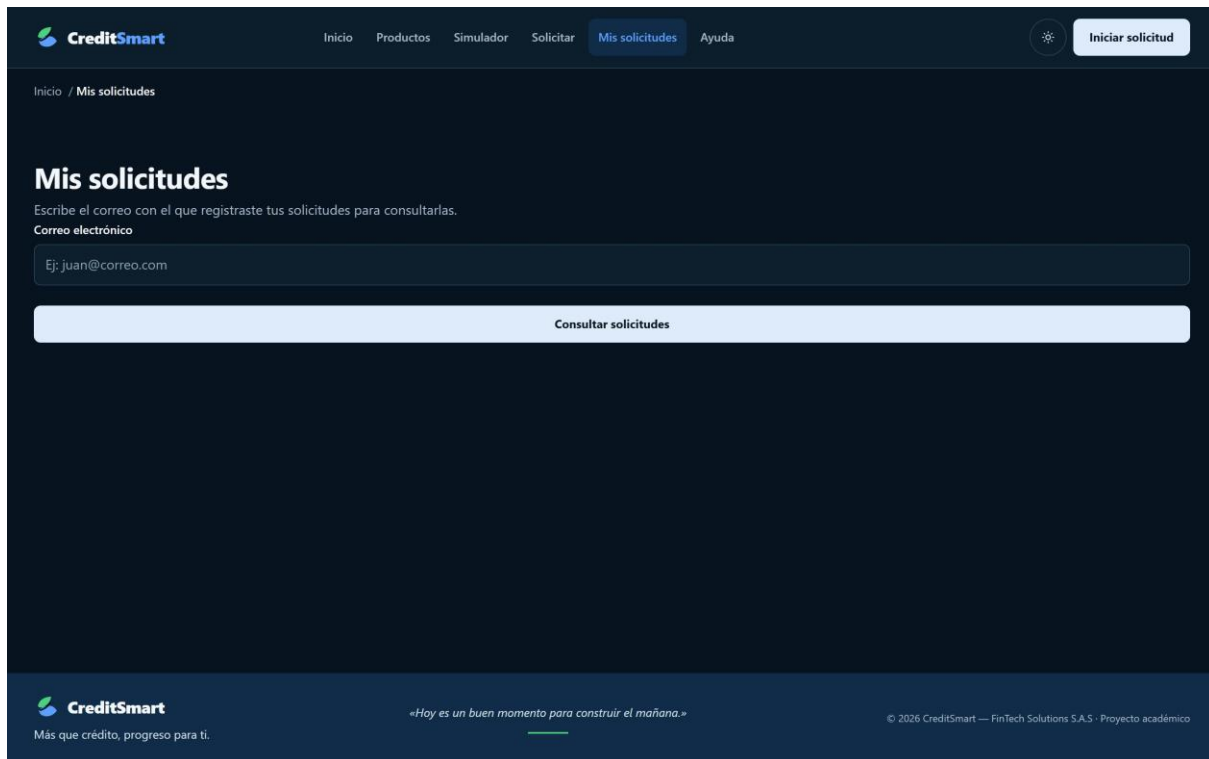


Figura 12. Estado inicial de «Mis solicitudes», antes de consultar.

10. Manejo de errores

Todas las operaciones con Firestore van dentro de try-catch. El repositorio traduce cualquier fallo en un error con mensaje claro; el caso de uso lo convierte en un Result de fallo; y el hook lo desenvuelve y lo muestra con un aviso y un estado de error en pantalla. Además, un tope de tiempo por operación convierte una desconexión de red en un error visible en lugar de un cuelgue: es lo que hace demostrable la prueba de «desconectar internet».

Capa	Paso	Qué ocurre
INFRAESTRUCTURA	Repositorio (Firestore)	try-catch: si falla la red, lanza un error con mensaje claro y registra el detalle en el logger.
APLICACIÓN	Caso de uso	Atrapa el error y devuelve Result.fromError; nunca lanza hacia la interfaz.
PRESENTACIÓN	Hook	Desenvuelve el Result, guarda el mensaje de error y avisa al usuario (toast).
PRESENTACIÓN	Página	Muestra el estado de error o el aviso; la app no se rompe.

Figura 13. Recorrido de un error desde Firestore hasta la interfaz

La degradación es la otra cara del manejo de errores: sin credenciales, el catálogo usa datos locales y las solicitudes quedan en memoria; si Firestore no responde para el catálogo, se sirve

el catálogo estático; si falta el índice compuesto, la consulta ordena en el cliente. Se degrada antes que fallar.

11. Seguridad — variables de entorno

Medida	Estado
Credenciales en .env	Sí, leídas con <code>import.meta.env.VITE_FIREBASE_*</code>
.env en .gitignore	Sí: .env, .env.local y .env.*.local excluidos
.env.example versionado	Sí, con las claves pero sin valores reales
Credenciales en el código o en git	No: el repositorio no contiene secretos
Configuración inyectada por constructor	Sí: ninguna clase lee un global

Tabla 3. Cumplimiento de seguridad de variables de entorno

12. Verificación

12.1 Comprobaciones automáticas

Comprobación	Comando	Resultado
Dominio y aplicación	<code>node tests/01-domain-application.mjs</code>	TODO OK
Arranque de la interfaz (7 rutas)	<code>node tests/02-boot-jsdom.mjs</code>	TODO OK
Empaquetado de producción	<code>npm run build</code>	Sin errores
Regla de dependencia (3 grep)	<code>grep` sobre domain, application y presentación</code>	Cero resultados

Tabla 4. Comprobaciones automáticas ejecutadas

La suite de arranque construye todo el grafo de dependencias (incluidos el cliente de Firebase y los repositorios de Firestore) y monta las siete rutas, entre ellas la nueva «Mis solicitudes», sin dejar errores en consola.

12.2 Comprobación de la conexión con Firestore

Se verificó la conexión real contra el proyecto con un script de humo que siembra los productos, crea una solicitud de prueba con `addDoc`, la consulta con `where + orderBy` y la borra. Es la misma secuencia que ejecuta la aplicación.

12.3 Comprobación en el navegador

- Cargar el catálogo y ver los productos desde Firestore.

- Registrar una solicitud y verla aparecer en la consola de Firebase.
- Consultar las solicitudes por correo en «Mis solicitudes».
- Ver el estado de carga durante las lecturas.
- Desconectar la red y comprobar que aparece un mensaje de error.



CRÉDITOS QUE TE IMPULSAN

Haz realidad tus planes

Compara, simula y solicita créditos de forma segura,
clara y 100% en línea.



Proceso
100% en línea



Simulación
inmediata



Tu información
protegida

Explorar créditos →



Soluciones de crédito para cada

Figura 14. La aplicación conserva su diseño responsive en móvil.

13. Cumplimiento de la rúbrica

Criterio	Pts	Evidencia en el proyecto	Auto
Configuración de Firebase	20	Proyecto y Firestore configurados; variables de entorno; .env en .gitignore (§4, §11)	
Operación READ — listar créditos	15	getDocs, estado de carga, try-catch, mapeo con id, render limpio (§7)	
Operación CREATE — guardar solicitudes	20	addDoc, validación previa, limpieza del formulario y redirección (§8)	
Consultas y filtros	15	where + orderBy; página «Mis solicitudes» filtrando por correo (§9)	
Manejo de errores	10	try-catch en todas las operaciones, mensajes claros, tope de tiempo (§10)	
Seguridad — variables de entorno	10	.env, .gitignore, .env.example, import.meta.env, sin secretos en git (§11)	
Video demostrativo	10	Guion en guion-sustentacion-EV3.docx; grabación por realizar	
Total	100		

Tabla 5. Criterios de la rúbrica y evidencia. La última columna queda para la autoevaluación

14. Entregables

14.1 Repositorio

<https://github.com/jossypaternina-max/creidibancos/tree/ev3-firebase>

14.2 Video sustentación

<https://youtu.be/mpT6T8f2ApQ>

Elemento	Estado
Código fuente completo con integración Firebase	Incluido en src/
.env.example (sin credenciales reales)	Incluido
.gitignore que incluye .env	Incluido
README.md actualizado	Sección 2.1: configuración de Firebase
Documentación de diseño	docs/24-integracion-firebase-ev3.md; índice en master.md
Video demostrativo (3-4 min)	Por grabar (obligatorio)
Firebase Console con las colecciones	Captura por añadir (§4.2)

Tabla 6. Contenido de la entrega

14.2 Historial de commits

La entrega se desarrolló en la rama ev3-firebase, a partir de la Actividad 2.

#	Commit	Qué aporta
1	feat: integra CreditSmart con Firebase Firestore (EV3)	Cliente de Firebase, repositorios de Firestore (CRUD con addDoc/getDocs/where+orderBy), página «Mis solicitudes», variables de entorno, manejo de errores y documentación

Tabla 7. Commit de la rama ev3-firebase

15. Conclusiones

- La integración con Firebase cubrió los seis criterios técnicos de la rúbrica: configuración y variables de entorno, lectura del catálogo con `getDocs`, creación de solicitudes con `addDoc`, consultas con `where` y `orderBy`, manejo de errores y seguridad de credenciales.
- La persistencia en la nube fue un cambio de adaptador. El dominio y los casos de uso no se modificaron, y la suite que los cubre siguió pasando sin ajustes. El único cambio de cableado vive en `config/dependencies.js`: dos líneas de repositorio y el registro del cliente de Firebase.
- Devolver siempre un `Result` desde los casos de uso y traducir los fallos de Firestore en mensajes claros permitió que el manejo de errores fuera uniforme en toda la aplicación, sin `try-catch` dispersos por la interfaz.
- La degradación —datos locales sin credenciales, catálogo estático si Firestore no responde, orden en cliente si falta el índice— hace que la aplicación sea robusta ante configuraciones incompletas y fallos de red, que es justamente lo que la actividad pide demostrar.
- La factory que reconstruye la entidad desde el documento de Firestore repite el patrón de capa anticorrupción de las actividades anteriores: si mañana cambia la forma del documento, solo cambia ese archivo.

16. Referencias

- Google. Firebase — Documentación oficial. firebase.google.com/docs
- Google. Cloud Firestore — Get started. firebase.google.com/docs/firestore/quickstart
- Google. Firestore — Add and manage data (CRUD). firebase.google.com/docs/firestore/manage-data/add-data
- Google. Firestore — Perform simple and compound queries. firebase.google.com/docs/firestore/query-data/queries
- Vite. Variables de entorno y modos. vite.dev/guide/env-and-mode
- Meta Open Source. React — Documentación oficial. react.dev
- Cockburn, A. Hexagonal Architecture (Ports and Adapters), 2005.
- Martin, R. C. Clean Architecture. Prentice Hall, 2017.
- Documentación interna: `docs/24-integracion-firebase-ev3.md` y `docs/master.md`.

Nota sobre el origen del contenido funcional: la aplicación es una reconstrucción del sitio de referencia `sweet-smart-credit-path.base44.app`. La arquitectura, la integración con Firebase, las validaciones y las pruebas son desarrollo propio.